NHR@SW I/O Training

DFTracer

Radita Liem
Johannes Gutenberg University Mainz

JOHANNES GUTENBERG UNIVERSITÄT MAINZ



Acknowledgements

Special thanks to **Hariharan Devarajan**, **Izzet Yildirim**, **Ray Sinurat** from Lawrence Livermore National Laboratory for providing their time and knowledge to improve this training

Projects

- DFTracer: https://github.com/llnl-asr/dftracer
- DFAnalyzer: https://github.com/llnl-asr/dfanalyzer
- dftracer-utils: https://github.com/llnl-asr/dftracer-utils
- pydftracer: https://github.com/llnl-asr/pydftracer

Agenda

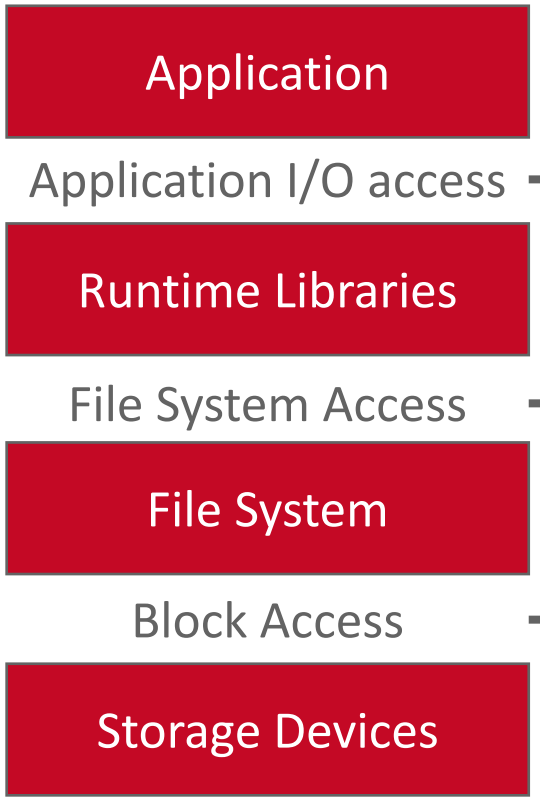
- **13:30 – 13:35 :** Welcome to Day 3
- **13:35 – 14:15 :** Introduction to DFTracer
- **14:15 – 15:00 :** Hands-on: Installation of DFTracer and BT-IO
- **15:00 – 15:30 :** Break
- **15:30 – 16:00 :** Hands-on: Using DFAnalyzer with BT-IO
- **16:00 – 16:55 :** Hands-on: Advanced DFTracer – Annotation & ML Workloads
- **16:55 – 17:00 :** Closing remarks / Recap day 3

Introduction to DFTracer

JOHANNES GUTENBERG UNIVERSITÄT MAINZ



I/O Measurement Strategy



Application I/O Tracing & profiling tools

- I/O Access expressed by the application
- Tools: Darshan, Extrae, Score-P, Tau, **DFTracer**

File System or POSIX Instrumentation

- I/O access patterns after they have been translated by I/O libraries such as MPI-IO or HDF5
- Tools: Lustre & GPFS Diagnostic, most tracing and profiling tools like Extrae, Tau, Score-P

Block Device Instrumentation

- Raw access to the disks
- Tools: Vendor specific tools, Linux block tracing

What is DFTracer? [1]

- C++ performance monitoring tool originally built for AI workloads.
- **DFTracer's default is Python-centric environment.** It can be installed through `pip` into your virtual environment including the rest of the DFTracer supporting tools
- DFTracer outputs JSON-formatted trace by default with `.pfw` or .pfw.gz extension
- Sample JSON raw output of DFTracer:

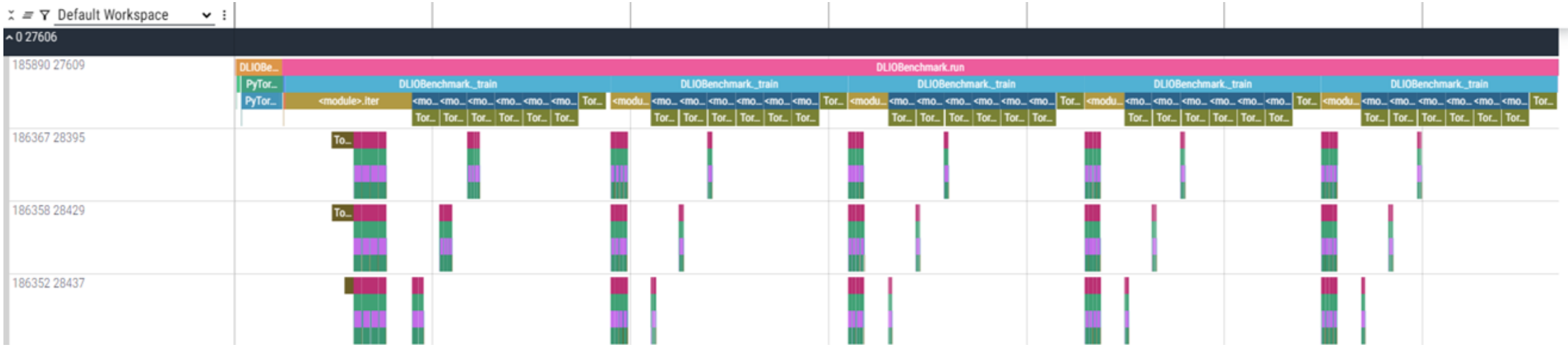

```
{"id":8,"name":"CUSTOM_BLOCK","cat":"CPP_APP","type":9,"pid":3308801,"tid":6617
602,"ts":1727286231145121,"dur":1000054,"ph":1,"args":{"hhash":39537,"p_idx":7,
"key":0,"level":3}}
```


- See more samples & details in: https://dftracer.readthedocs.io/en/latest/trace_format.html

What is DFTracer? [2]

- DFTracer also works on typical HPC C and C++ applications
- DFTracer's default is tracing that provides granular details of the I/O activities inside the application phases.

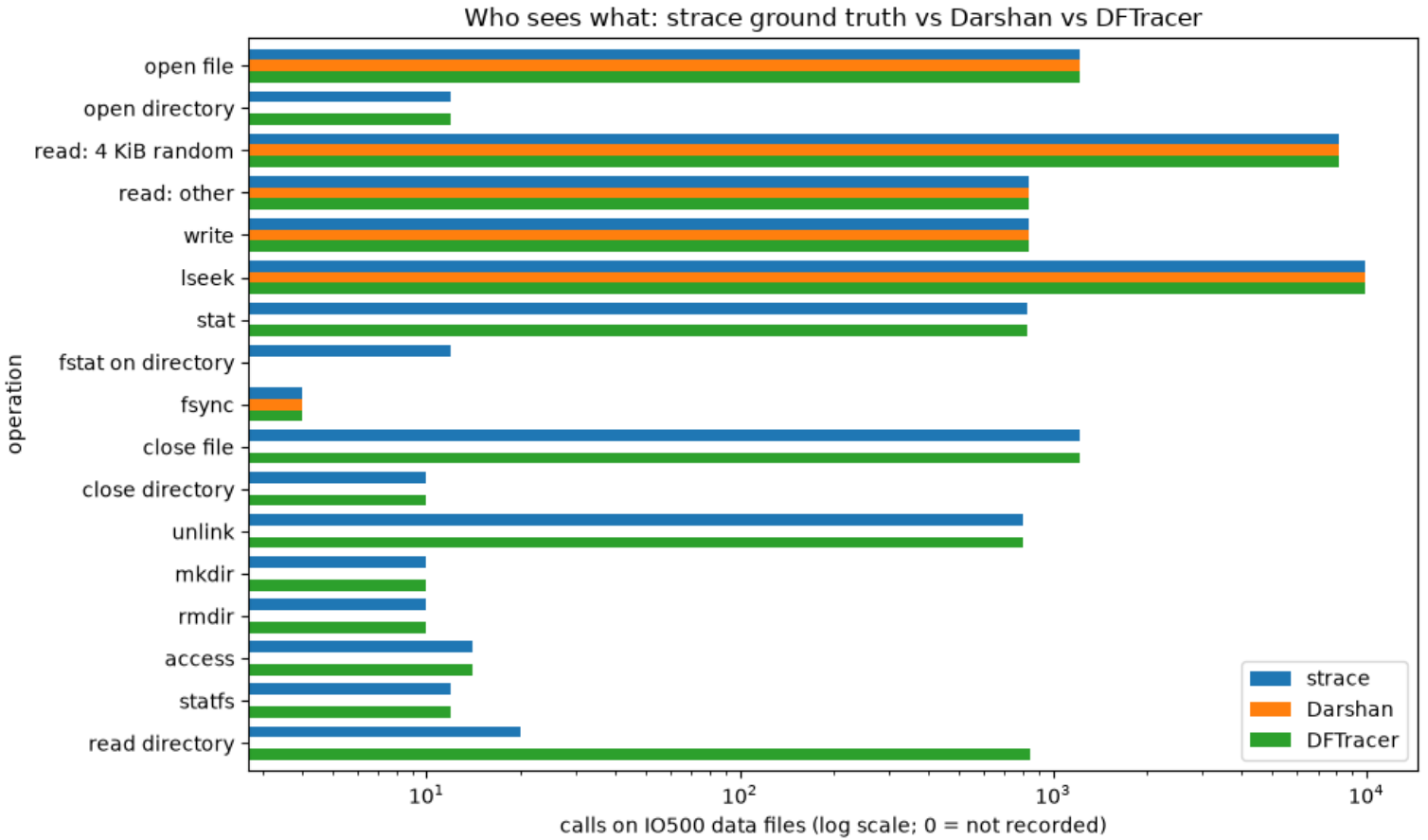
  The figure below is a sample trace of DFTracer on DLIO benchmark run using Pytorch[1]. It shows from the top where it shows the overall run followed with training phases, further down to the Torch functions until the POSIX I/O operations inside



[1] D. Djebarov et al., "Streamlining HDF5's AI Workloads Benchmarking," IEEE IPDPSW, pp. 722–730, 2025.

Why Using DFTracer?

- It works in the same environment as typical HPC AI workloads: **Python**
- Robust metadata coverage with low tracing overhead:



DFTracer Training Setup

- The training will be divided into two parts: **running applications in the cluster** and **offline post-mortem analysis**

- **Part 1: Application Execution (HPC Cluster)**
  - Install DFTracer
  - Run DFTracer on NAS parallel benchmark – BT-IO and DLIO benchmark
  - Collecting and compacting the traces
  - Basic analysis using DFAnalyzer

- **Part 2: Offline Post-Mortem Analysis (Local Machine)**
  - Visualize collected traces using DFTracer visualization server.
  - Perform advanced data analysis in Jupyter notebook

Installing & Running DFTracer

JOHANNES GUTENBERG UNIVERSITÄT MAINZ



Preparing Virtual Environment in MOGON

- DFTracer is Python-based, it needs to be installed with Python pip

```
# adding necessary modules
module load mpi/OpenMPI/5.0.3-GCC-13.3.0
module load lang/Python/3.12.3-GCCcore-13.3.0

# creating a virtual environment called dftracer-training
python -m venv dftracer-training

# activate the virtual environment
source dftracer-install/bin/activate
```

Installing DFTracer in Virtual Environment

- Checking the right pip and python, upgrade pip, and install DFTracer

```
# ensuring that pip and python come from the virtual environment
which pip
which python

# upgrade pip
pip install --upgrade pip

# install DFTracer and DFAnalyzer
pip install --pre dftracer[dfanalyzer]
```

Preparing NAS Parallel Benchmark – BT-IO

- Preparing the test application installation using NAS parallel benchmark

```
# download the source
wget https://www.nas.nasa.gov/assets/npb/NPB3.4.4.tar.gz

# unpack NAS parallel benchmark tar.gz
tar -zxvf NPB3.4.4.tar.gz

# install NAS parallel benchmark
cd NPB3.4.4/NPB3.4-MPI && cp config/make.def.template config/make.def

# build the benchmark
make bt CLASS=C SUBTYPE=full
```

Attaching DFTracer to Slurm Script

- DFTracer is meant to be used with in-code annotation, therefore several mandatory environment variables are needed

```
# DFTracer needs to be enabled explicitly in case there's an annotation inside
that will be triggered during normal run
export DFTRACER_ENABLE=1


# set the mode for the DFTracer. PRELOAD collects everything inside the
application
export DFTRACER_INIT=PRELOAD


# set where the log of DFTracer will be stored and the prefix name
export DFTRACER_LOG_FILE=[output directory]/[trace prefix name]
```

Sample Slurm Script [1]

- Copy the Slurm script inside `/lustre/project/nhr-hpckurs/DFTracer` to your own directory

```
. . .
#SBATCH --account=nhr-hpckurs
#SBATCH --partition=smallcpu
#SBATCH --time=00:15:00
#SBATCH --job-name=NPB-test
#SBATCH --output=NPB-%j.txt

# load necessary modules
module load mpi/OpenMPI/5.0.3-GCC-13.3.0
module load lang/Python/3.12.3-GCCcore-13.3.0
. . .
```

Sample Slurm Script [2]

- Make sure `LD_PRELOAD` and the application is pointing to the right place

```
. . .
# add DFTracer variables
export DFTRACER_ENABLE=1
export DFTRACER_INIT=PRELOAD
export LD_PRELOAD=[your own directory]/dftracer-training/lib64/python3.12/site-packages/dftracer/lib64/libdftracer_preload.so

# run NAS parallel benchmark
mpiexec -n 9 [your own directory]/NPB3.4.4/NPB3.4-MPI/bin/bt.C.x.mpi_io_full
```

Expected Output

- After you run the job with **`sbatch npb-slurm.sh`** this is the output to expect:

```
(io-training) hpcgst0777@mogon-nhr01:/lustre/project/nhr-hpckurs/DFTracer$ ls
NPB-1471417.txt                                trace-5f20bf60743f07d9-preload.pfw.gz
NPB3.4.4                                       trace-8189eee1561a281f-fork.pfw.gz
NPB3.4.4.tar.gz                                trace-911c69540162c2de-fork.pfw.gz
btio.full.out                                  trace-b830eb96adef8e40-preload.pfw.gz
dlio_benchmark                                 trace-ba70c7253bbd22c2-fork.pfw.gz
io-training                                    trace-bf6caab2b0560664-preload.pfw.gz
npb-slurm.sh                                   trace-c46bb32f7cd39770-fork.pfw.gz
trace-0dc9c1da2597da55-preload.pfw.gz          trace-c78a4c334c5266bb-preload.pfw.gz
trace-0f5abcac403a8874-preload.pfw.gz          trace-cdfb9584f2715fe0-preload.pfw.gz
. . .
```

Analyzing DFTracer Output

JOHANNES GUTENBERG UNIVERSITÄT MAINZ



Compacting Traces and Running DFAnalyzer

- DFTracer output can be analyzed post-mortem using various tools such as DFAnalyzer
- Since DFTracer outputs are fragmented, compaction is needed to improve analysis performance:

```
# compacting traces with dftracer_split
dftracer_split -n bt.C.x.mpi_io_full -c --verify
```

- The compacted output can be analyzed with DFAnalyzer to get an overview

```
# run DFAnalyzer
dfanalyzer \
  trace_path=[your own directory]/dftracer-split \ # output directory name
  analyzer/preset=posix \ # there's two presets: posix and ai
  hydra.run.dir=[your own directory]/dfanalyzer-output/bt.C.x.mpi_io_full
  # preservation of the running configuration done by Hydra, a Python library
```

DFAnalyzer Default Output

- What you get from running the previous DFAnalyzer command:

```
✓ Cluster setup (0.013s)
✓ Analyzer setup (0.015s)
✓ Build index (9.138s)
✓ Read trace & stats (1.212s)
✓ Compute high-level metrics (1.027s)
✓ Compute views (2.402s)
✓ Process views (0.211s)
```

Time Period Summary

Metric	Unit	Value
Job Time	seconds	124.591
Trace Count	count	8,014
Profile Count	count	0
Total Count	count	8,014
Total Files	count	56
Total Nodes	count	1
Total Processes	count	10
POSIX - All Count	count	7,225
POSIX - All Size	MB	12973.832
POSIX - All Bandwidth	MB/s	1733.818
POSIX - All Avg Transfer Size	MB	1.796

Layer Breakdown

Layer	Time (s)	Ops	Ops/sec	Size (MB)	Bandwidth (MB/s)
POSIX - All	7.483	7,225	965.546	12973.832	1733.818

```
✓ Output (0.013s)
✓ Cluster teardown (0.000s)
```

Summary interpretation details:
https://dftracer.readthedocs.io/projects/dfanalyzer/en/latest/interpreting-summary.html

Visualizing DFTracer Traces

- DFTracer has its own visualization server that can be activated by running:

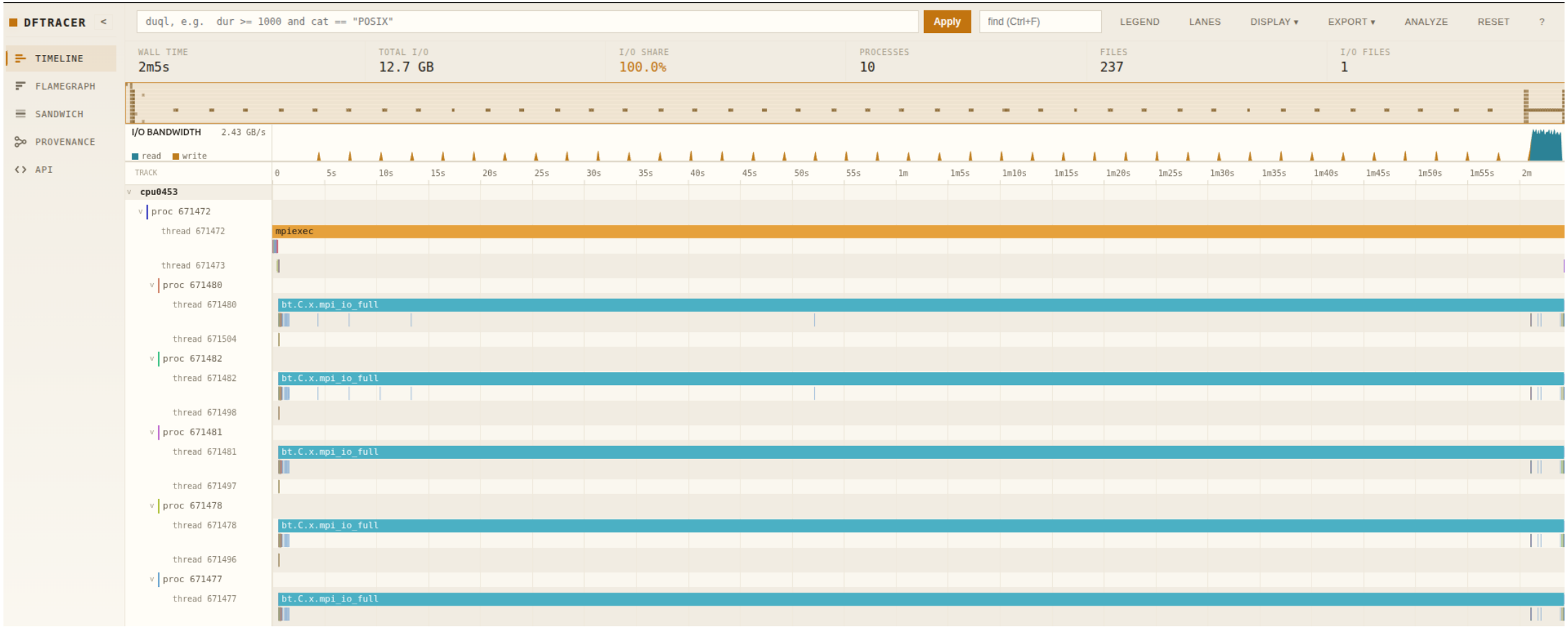
```
# running the visualization server
dftracer_server -d [directory created by the dftracer_split]
```

- For this training, MOGON cannot facilitate running the DFTracer server. Therefore, the trace needs to be downloaded to the local environment.

```
# scp command if you don't set your config file
scp -J [username]@hpcgate.zdv.uni-mainz.de [username]@mogon-nhr-01:[path] .
```

DFTracer Visualization Server – BT-IO

- The visualization output of NAS parallel benchmark from the cluster



DLIO with DFTracer

JOHANNES GUTENBERG UNIVERSITÄT MAINZ

What is DLIO?

- **Open-source I/O benchmark for deep learning workloads.**

  DLIO is also part of MLPerf Storage. Benchmark to evaluate storage for machine learning workloads (https://github.com/mlcommons/storage)

- **DLIO tries to emulates I/O behavior of real deep learning applications.**

  It already provides several workload configurations of some popular machine learning workloads that can be found in: https://github.com/argonne-lcf/dlio_benchmark/tree/main/dlio_benchmark/configs/workload

- Simulation inside DLIO covers: **data loading**, **training,** and **checkpointing.**

  Users can create their own configuration using the yaml file to configure the datasets, file formats, I/O patterns, and the backend (PyTorch or Tensorflow)

Installing DLIO

- DLIO benchmark in this training also can be found in `/lustre/project/nhr-hpckurs/DFTracer`

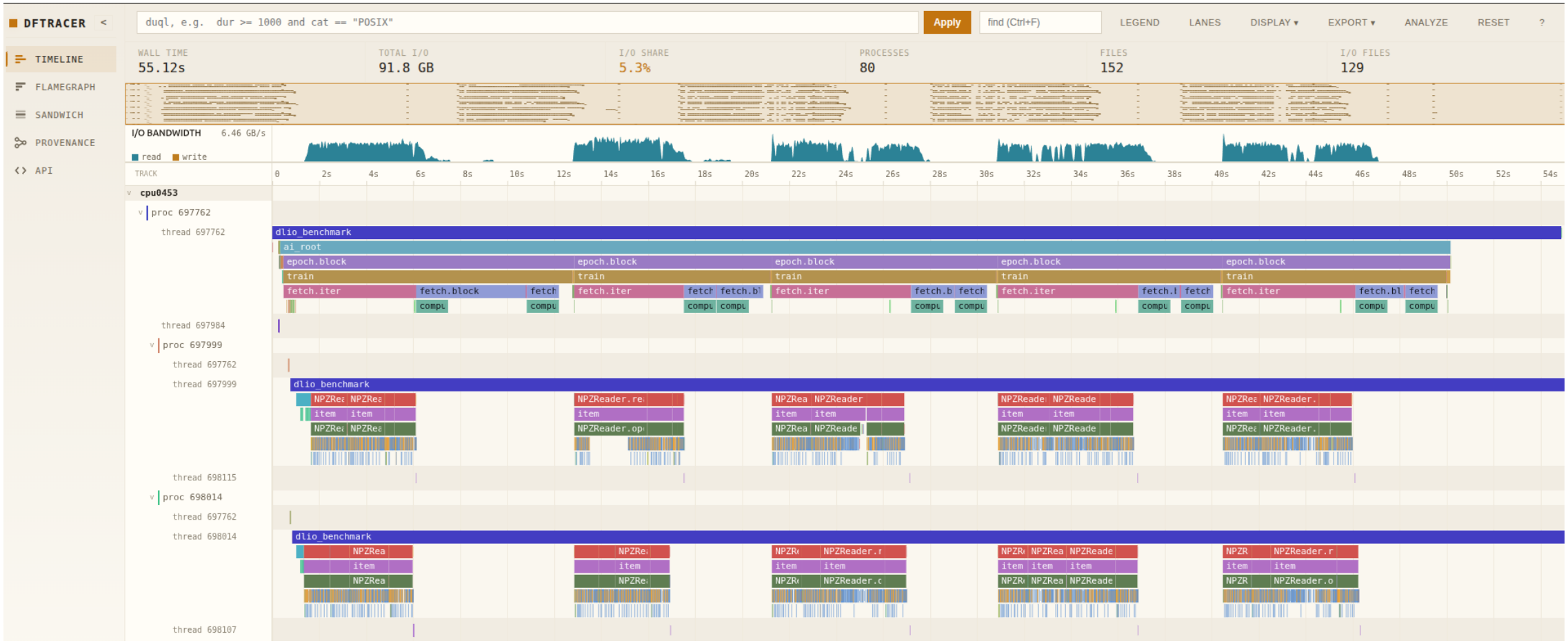
```
# clone the DLIO benchmark
git clone https://github.com/argonne-lcf/dlio_benchmark

# enter the benchmark and install
cd dlio_benchmark/
pip install .
```

- More details of the installation and the structure of the benchmark can be found in their documentation: https://dlio-benchmark.readthedocs.io/en/latest/

DFTracer Visualization Server – DLIO

- The visualization output of DLIO – UNet3D workload from the cluster



DFTracer Annotation

JOHANNES GUTENBERG UNIVERSITÄT MAINZ

DFTracer Modes

- DFTracer can be used in three main modes:
  - **App mode:** DFTracer will show data on certain regions of the applications.
  - **Preload mode:** DFTracer will profile POSIX or STDIO calls for the whole run.
  - **Hybrid mode:** Combination of the app and preload mode.

- These modes will be realized in **`DFTRACER_INIT`** using **`FUNCTION`** for the **app mode** and **`PRELOAD`** for the preload mode. Hybrid Mode uses **`PRELOAD`**

- The default for **`DFTRACER_INIT`** is **`FUNCTION`**
- More information on the DFTracer modes can be seen here: https://dftracer.readthedocs.io/en/latest/api.html

In-Code Annotation using DFTracer [1]

- DFTracer's default is actually for performing code annotation. The annotation can be done in the Python application and also in the C or C++ program

  Some Python APIs for annotations:

```
# import dftracer library
from dftracer.python import dftracer

# initialize DFTracer
dft_logger = dftracer.initialize_log(logfile, data_dir, process_id)

...

# finalization (optional)
dft_logger.finalize()
```

In-Code Annotation using DFTracer [2]

- APIs for C code annotations

```
# adding header
#include <dftracer/dftracer.h>

# initialize DFTracer
DFTRACER_C_INIT(log_file, data_dirs, process_id);

...

# finalization
DFTRACER_C_FINI();
```

In-Code Annotation using DFTracer [3]

- APIs for C++ code annotations

```
# adding header
#include <dftracer/dftracer.h>

# initialize DFTracer
DFTRACER_CPP_INIT(log_file, data_dirs, process_id);

...

# finalization
DFTRACER_CPP_FINI();
```

Advanced Materials

JOHANNES GUTENBERG UNIVERSITÄT MAINZ

JG|U

MPI and HDF5 Support for DFTracer Installation

- The current DFTracer installation setup doesn't have MPI and HDF5 library enabled. This is due to large varieties of MPI and HDF5 functions out there so the exact MPI and HDF5 need to be specified.

  More information: https://dftracer--407.org.readthedocs.build/en/407/build.html#auto-detect-with-pip

```
# set the DFTRACER_ENABLE_MPI and DFTRACER_ENABLE_HDF5
export CC=$(which mpicc) CXX=$(which mpicxx)
export DFTRACER_ENABLE_MPI=ON DFTRACER_ENABLE_HDF5=ON
pip install --no-cache-dir --no-binary=dftracer dftracer.
```

- Currently DFTracer supports OpenMPI, Cray MPI and MPICH

Thank you!

If you have any question: **Radita Liem** (rliem@uni-mainz.de)

JOHANNES GUTENBERG
UNIVERSITÄT MAINZ

JG|U